Course Management

Vizsga dokumentáció

Spring Boot alapú kurzuskezelő rendszer

Projekt neve	Course Management System
Technológia	Java 17, Spring Boot 3.5, Maven
Adatbázis	MySQL (normál futás) + H2 (teszt profil)
Tesztelés	JUnit 5, Mockito, MockMvc
Készítette	Hallgatói vizsgaprojekt

1. A projekt célja

A projekt célja egy olyan Spring Boot alapú kurzuskezelő rendszer megvalósítása, amely kezeli a felhasználókat, a kategóriákat és a kurzusokat, továbbá támogatja a hallgatók kurzusra történő feliratkoztatását. Az alkalmazás réteges architektúrában készült, REST API-t biztosít, valamint külön frontend résszel és tesztkörnyezettel is rendelkezik.

- kurzusok létrehozása, lekérdezése, módosítása és törlése
- felhasználók és kategóriák kezelése
- oktató hozzárendelése kurzushoz
- hallgatók feliratkoztatása kurzusokra
- validáció, kivételkezelés és naplózás alkalmazása
- unit és integration tesztek biztosítása

2. Felhasznált technológiák

Terület	Megoldás
Backend	Java 17, Spring Boot, Spring Web, Spring Data JPA
Adatbázis	MySQL normál futáshoz, H2 in-memory adatbázis tesztekhez
Segédkönyvtárak	Hibernate, Lombok, Validation API
Frontend	HTML, CSS, JavaScript (Spring Boot static erőforrások)
Tesztelés	JUnit 5, Mockito, MockMvc
Eszközök	Maven, Postman, Git, GitHub

3. Projektstruktúra

A projekt főbb package-ei és szerepük a következők:

```
src/main/java/com/example/course_management
├─ controller
│  ├─ CourseController.java
│  ├─ UserController.java
│  └─ CategoryController.java
├─ service
│  ├─ CourseService.java
│  ├─ UserService.java
│  └─ CategoryService.java
├─ repository
│  ├─ CourseRepository.java
│  ├─ UserRepository.java
│  └─ CategoryRepository.java
├─ entity
│  ├─ Course.java
│  ├─ User.java
│  └─ Category.java
├─ dto
│  ├─ CourseCreateRequest.java
│  ├─ CourseResponse.java
│  ├─ UserCreateRequest.java
```

```
│   ├── UserResponse.java
│   ├── CategoryCreateRequest.java
│   └── CategoryResponse.java
├── mapper
│   ├── CourseMapper.java
│   ├── UserMapper.java
│   └── CategoryMapper.java
└── exception
    ├── ResourceNotFoundException.java
    ├── ErrorResponse.java
    └── GlobalExceptionHandler.java
```

A frontend a src/main/resources/static mappában található, a tesztek pedig a src/test/java alatt helyezkednek el.

4. Entitások és kapcsolatok

A rendszer három fő entitást használ: User, Course és Category. A követelményeknek megfelelően megtalálható benne legalább egy Many-to-One és legalább egy Many-to-Many kapcsolat.

- Course → User (Many-to-One): minden kurzushoz egy oktató tartozik.
- Course ↔ User (Many-to-Many): egy kurzusra több hallgató iratkozhat fel, és egy hallgató több kurzuson is részt vehet.
- Course ↔ Category (Many-to-Many): egy kurzus több kategóriába sorolható, és egy kategóriába több kurzus tartozhat.

Főbb mezők

Entitás	Mezők
User	id, fullName, email
Category	id, name
Course	id, title, description, durationInHours, instructor, categories, students

5. REST API végpontok

HTTP	Végpont	Leírás
POST	/api/users	Új felhasználó létrehozása
GET	/api/users	Összes felhasználó lekérdezése
POST	/api/categories	Új kategória létrehozása
GET	/api/categories	Összes kategória lekérdezése
POST	/api/courses	Új kurzus létrehozása
GET	/api/courses	Összes kurzus lekérdezése
GET	/api/courses/{id}	Kurzus lekérdezése azonosító alapján
PUT	/api/courses/{id}	Kurzus módosítása
DELETE	/api/courses/{id}	Kurzus törlése

POST	/api/courses/{courseId}/enroll/{studentId}	Hallgató feliratkoztatása a megadott kurzusra

6. DTO minták

CourseCreateRequest

```
{
  "title": "Spring Boot Basics",
  "description": "Introduction to Spring Boot",
  "durationInHours": 20,
  "instructorId": 1,
  "categoryIds": [1, 2],
  "studentIds": [2]
}
```

UserCreateRequest

```
{
  "fullName": "John Doe",
  "email": "john@test.com"
}
```

CategoryCreateRequest

```
{
  "name": "Programming"
}
```

A DTO-k használata azért fontos, mert a kliens és a backend közötti adatcsere elkülönül az entitások belső szerkezetétől.

7. Frontend összefoglaló

A frontend a Spring Boot alkalmazás static mappájából kiszolgált HTML, CSS és JavaScript fájlokra épül. A felület dashboard-szerű elrendezést használ, és a következő funkciókat biztosítja:

- felhasználók létrehozása és listázása
- kategóriák létrehozása és listázása
- kurzus létrehozása oktató- és hallgatóválasztással
- hallgatók feliratkoztatása kurzusra
- kurzusok listázása és törlése
- statisztikai kártyák és siker/hiba visszajelzések

8. Postman tesztelés

A REST API végpontok tesztelése Postman segítségével történt. A projekt tartalmaz exportált Postman kollekciót, amely a fontosabb CRUD műveleteket, valamint a hibás bemenetek vizsgálatát is lefedi.

- felhasználó létrehozás és lekérdezés
- kategória létrehozás és lekérdezés
- kurzus létrehozás, módosítás és törlés
- hallgató feliratkoztatása kurzusra
- validációs hibák ellenőrzése

Postman teszt

Home Workspaces API Network

Search Postman Ctrl K

Invite Upgrade

Balázs Kovács's Workspace New Import

Collections Environments History Flows

Search collections

Course Management

Users

POST Create User - John Doe

POST Create User - Peter Smith

GET Get All Users

POST Create Invalid User

Categories

POST Create Category - Programming

POST Create Category - Backend

GET Get All Categories

POST Create Invalid Category

Courses

POST Create Course - Spring Boot Ba...

GET Get All Courses

GET Get Course By Id

PUT Update Course

POST Enroll Student To Course

POST Create Invalid Course

DEL Delete Course

Course Management - Run results

Ran today at 02:39:53 PM · View all runs

Run Again New Run Automate Run Share

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	2s 130ms	0	0	195 ms

All Tests Passed (0) Failed (0) Skipped (0) Errors (0) Console Log

View Summary

Iteration 1

POST Users / **Create User - John Doe**
http://localhost:8080/api/users
200 • 612 ms • 307 B
No tests found

POST Users / **Create User - Peter Smith**
http://localhost:8080/api/users
200 • 11 ms • 311 B
No tests found

GET Users / **Get All Users**
http://localhost:8080/api/users
200 • 127 ms • 368 B
No tests found

POST Users / **Create Invalid User**
http://localhost:8080/api/users
400 • 28 ms • 331 B
No tests found

Online Find and replace Console Postbot Runner Start Proxy Cookies Vault Trash

Home Workspaces API Network

Search Postman Ctrl K

Invite Upgrade

Balázs Kovács's Workspace New Import

Collections Environments History Flows

Search collections

Course Management

Users

POST Create User - John Doe

POST Create User - Peter Smith

GET Get All Users

POST Create Invalid User

Categories

POST Create Category - Programming

POST Create Category - Backend

GET Get All Categories

POST Create Invalid Category

Courses

POST Create Course - Spring Boot Ba...

GET Get All Courses

GET Get Course By Id

PUT Update Course

POST Enroll Student To Course

POST Create Invalid Course

DEL Delete Course

Course Management - Run results

Ran today at 02:41:01 PM · View all runs

Run Again New Run Automate Run Share

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	1s 501ms	0	0	12 ms

All Tests Passed (0) Failed (0) Skipped (0) Errors (0) Console Log

View Summary

Iteration 1

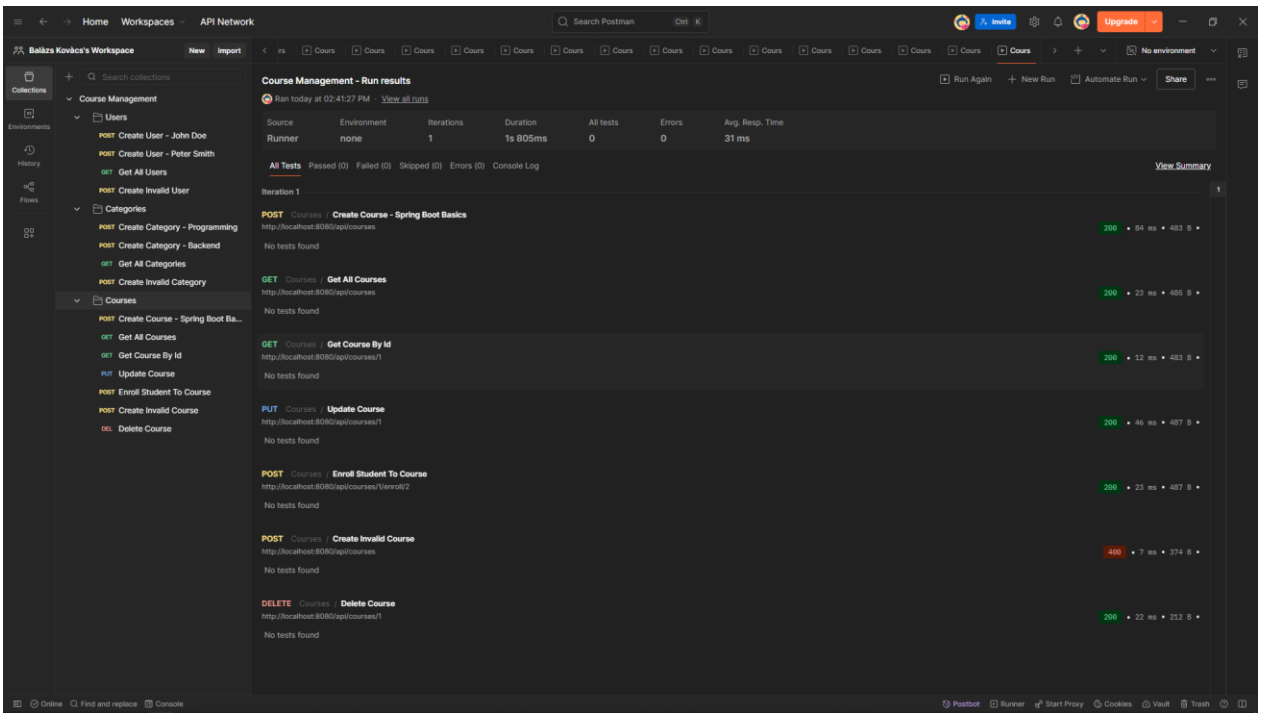
POST Categories / **Create Category - Programming**
http://localhost:8080/api/categories
200 • 19 ms • 282 B
No tests found

POST Categories / **Create Category - Backend**
http://localhost:8080/api/categories
200 • 11 ms • 278 B
No tests found

GET Categories / **Get All Categories**
http://localhost:8080/api/categories
200 • 8 ms • 310 B
No tests found

POST Categories / **Create Invalid Category**
http://localhost:8080/api/categories
400 • 8 ms • 331 B
No tests found

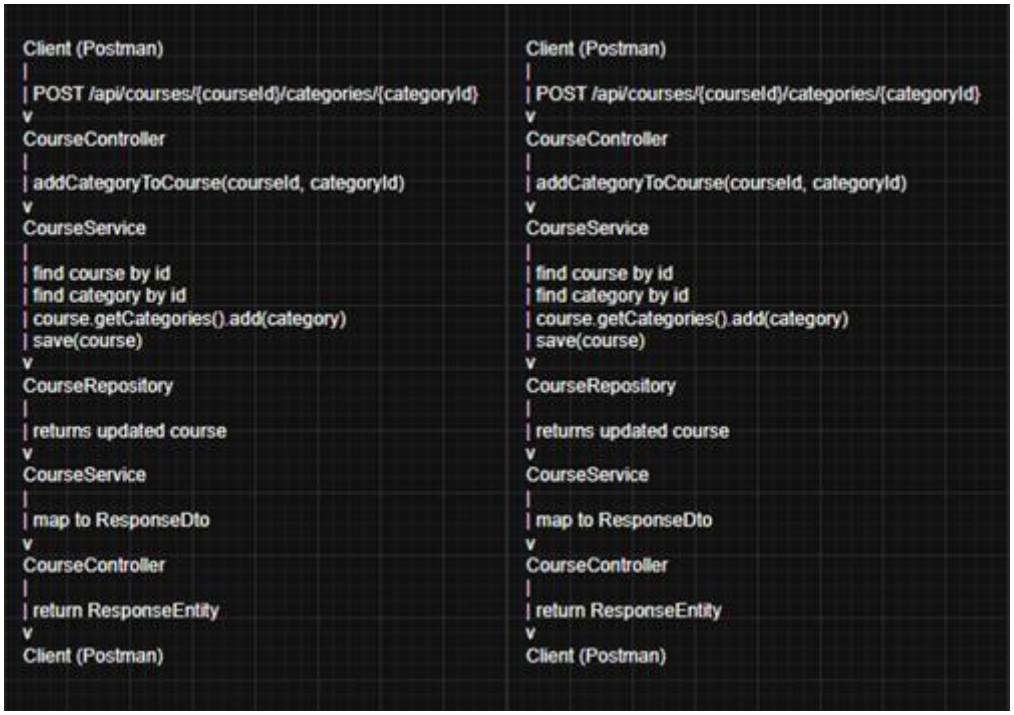
Online Find and replace Console Postbot Runner Start Proxy Cookies Vault Trash



9. UML és adatbázis diagramok

A feladathoz a követelményeknek megfelelően két UML diagram és egy ER / adatbázisterv is készült. Ezek a dokumentációhoz és a docs mappához egyaránt csatolhatók.

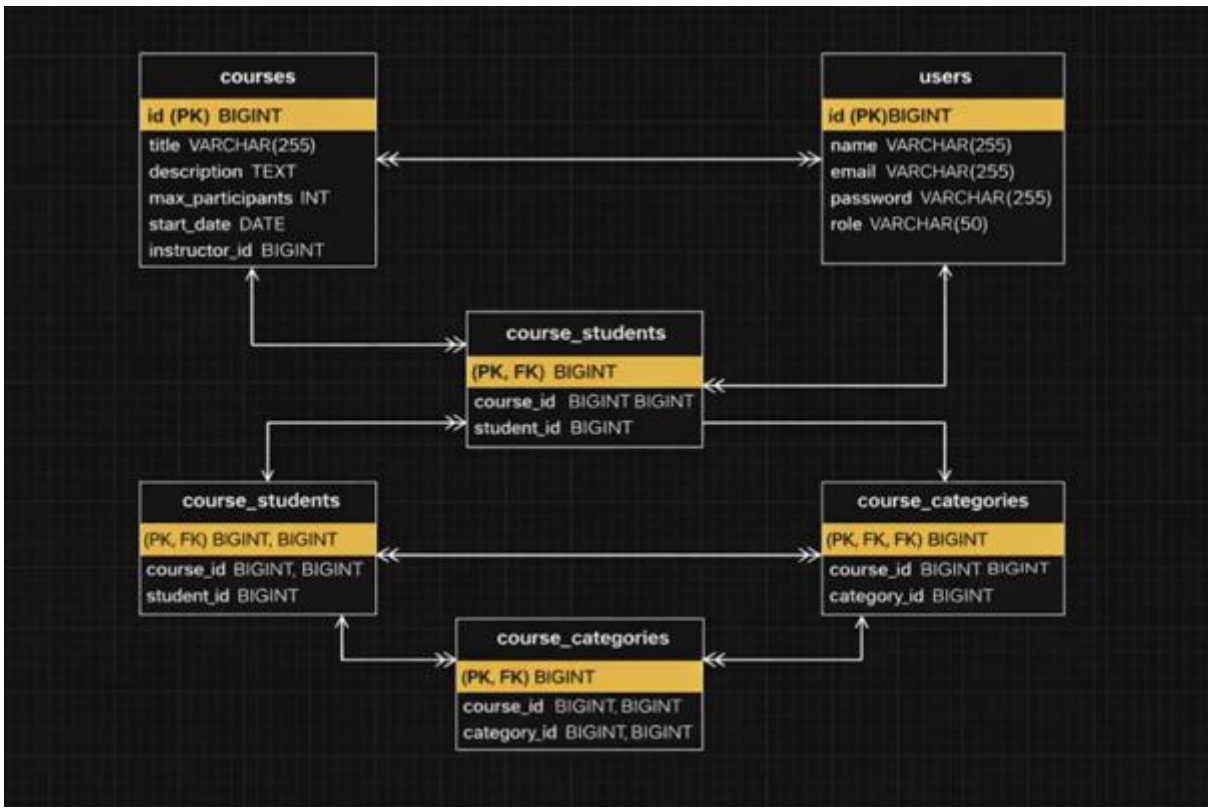
Class diagram



Client (Postman)
| POST /api/courses/{courseId}/categories/{categoryId}
v
CourseController
| addCategoryToCourse(courseId, categoryId)
v
CourseService
| find course by id
| find category by id
| course.getCategories().add(category)
| save(course)
v
CourseRepository
| returns updated course
v
CourseService
| map to ResponseDto
v
CourseController
| return ResponseEntity
v
Client (Postman)

Sequence diagram

ER diagram



10. Tesztelés

A projekt unit és integration teszteket is tartalmaz. A unit tesztek a service réteg logikáját vizsgálják, míg az integration tesztek H2 adatbázissal és MockMvc segítségével ellenőrzik a vezérlőréteg működését.

Teszt típusa	Megvalósítás
Unit tesztek	CourseServiceTest, UserServiceTest, CategoryServiceTest – Mockito + JUnit 5
Integration tesztek	CourseControllerIntegrationTest, UserCategoryControllerIntegrationTest – SpringBootTest + MockMvc + H2
Validáció	@NotBlank, @NotNull, @Min, @Email annotációk a DTO-kon
Kivételkezelés	ResourceNotFoundException és GlobalExceptionHandler

11. Futtatás és konfiguráció

Normál futás

```
mvn spring-boot:run
```

Tesztek futtatása

```
mvn test
```

Adatbázis hitelesítési adatok

A MySQL kapcsolat a dokumentált környezeti változókon keresztül konfigurálható. A projektben a DB_USERNAME és DB_PASSWORD változók használhatók, alapértelmezett fejlesztői értékkel.

```
spring.datasource.username=${DB_USERNAME:root}
spring.datasource.password=${DB_PASSWORD:root}
```

12. Összegzés

A Course Management System projekt teljesíti a feladatkiírás főbb elvárásait: Spring Boot alapú backenddel, MySQL/H2 adatbázis-kezeléssel, DTO-k használatával, CRUD REST API-val, frontenddel, Postman tesztekkel, unit és integration tesztekkel, valamint UML és ER diagramokkal rendelkezik. A projekt réteges szerkezete, a validáció, a kivételkezelés és a logolás egyaránt a tiszta, átlátható megvalósítást támogatják.